

SECURITY PLAN

Production Readiness Assessment

COMPASS — nine axes, measured 13 August 2026

DATE

13 August 2026

SCOPE

Webapp, backend, AI pipeline, and the operational layer

TARGET HOST

caire-server (10.226.8.205)

DATA CLASSIFICATION

Real PHI — HIPAA in scope

ASSESSMENT ONLY. Nothing here has been executed, and no finding has been fixed.

Context

The system currently serves real patient data while configured as a research deployment. This document answers one question: **what is missing before this can be called production?**

An earlier draft of this file covered security alone. That was too narrow. Security is one of nine axes assessed here, and it is not the one that fails first — **an unattended reboot would silently stop the entire system today**, and no TLS certificate would change that.

Every finding below was verified by direct observation on 2026-08-13. Where a number appears, it was measured, not estimated.

Relationship to existing documents

- `SECURITY_AUDIT.md` (2026-02-12) is largely invalid. It assumes nginx plus a Dockerised backend; the deployment is now native uvicorn plus a Docker webapp, with no nginx anywhere. This document supersedes it.
- `PHI_COMPLIANCE.md` remains the governing policy reference (Cedars-Sinai AI Compliance Policy + HIPAA) and is cited where relevant.
- `docs/setup/DEPLOYMENT_*.md` describe how to stand the system up. They do not cover how to keep it up, which is most of what follows.

Readiness scorecard

#	Axis	Verdict	The single worst item
A	Release & deployment	Blocker	No process supervision — a reboot stops everything, permanently
B	CI & quality gates	Blocker	Nightly E2E has failed 60 of 60 runs since 2026-06-16
C	Observability	Blocker	No error tracking, no metrics, no alerting of any kind
D	Reliability	Gap	Every component is a single point of failure with no restart
E	Data & backup	Blocker	No database backup exists
F	Frontend readiness	Gap	No error boundary — patients see the raw Next.js crash page
G	Testing	Gap	Good unit coverage, but the E2E safety net is dead

#	Axis	Verdict	The single worst item
H	Docs & runbooks	Gap	No runbook, incident, or rollback procedure
I	Security	Blocker	No TLS; <code>/docs</code> open on the LAN; no PHI audit log
J	Access logging & retention	Blocker	The access log records the proxy's IP for 93% of requests, and lives in <code>/tmp</code>

Blocker = must be resolved before production. **Gap** = required for a system that stays healthy, but not a launch stopper. **OK** = adequate as-is.

One paragraph, no jargon

This system is a **prototype that works well in the lab**. It is handling real patient data, so it needs to become **equipment that runs safely in a hospital for years**. The gap between those two things is what this document describes.

By analogy: the car is built and it drives. But **it does not restart itself when the engine cuts out** (axis A), **there is no dashboard, so the driver cannot tell something is wrong** (axis C), **there is no dashcam, so after an incident nobody can establish what happened** (axis J), and **there is no spare tyre** (axis E, backup). The seatbelt is missing too (axis I, security) — but fitting only the seatbelt fixes none of the rest.

Each axis below opens with a **"Why this matters"** note written without technical vocabulary, describing what actually goes wrong if it is left alone. Reading only those notes is enough to judge the priorities.

System architecture

The deployment as it actually runs, measured 2026-08-13. Markers: [!] Blocker, [~] needs work, [+] adequate.

patient · doctor
· admin

Browser

[!] plain HTTP, no TLS

LAN 10.226.8.205

auto-closes after 8/15

[+] auto-restarts

(1) Webapp — Docker container

[~] no error screen

prostatecancer-webapp-native

Next.js 13 · 0.0.0.0:3001→3000

up 5 days · healthy · 0 restarts

[!] drops client IP

(2) API proxy (route.ts)

no X-Forwarded-For

host.docker.internal

(seen as container 172.31.0.2)

[!] unsupervised PPID1

(3) Backend — FastAPI (native)

[!] logs under /tmp

PID 2117291 · up 12d16h

[!] /docs on the LAN

0.0.0.0:18001 · 3 workers

[+] loopback

(4) PostgreSQL

(5) Redis

(6) NLP

[!] no backup

127.0.0.1

127.0.0.1

gateway

:5439

:6380

0.0.0.0

17 tables

[~] no

:18080

auth

[!] unsup.

127.0.0.1:8888

[+] auto-rest.

(7) NLP

classifier 2wk

(8) AI pipeline watcher

PID 1438747 · 13 days

[!] unsupervised PPID1

(9) External services

Azure OpenAI

REDCap

```

| polls drop folder /5s |           | [~] no failure UX |
|-----▲-----|
| files dropped in |
|-----|
| (10) De-identification app |
| dist/ built 24 July |
| [!] source drift |
| [!] deid_mapping.csv |
| line 2 corrupt |
|-----|

```

Per component — what is missing, and what makes it production grade

#	Component	What is missing today	What makes it production grade
1	Webapp container	A render error shows patients the developer crash page. 99 component files, 151k lines, with 62 stale versions alongside	Add <code>error.tsx</code> / <code>not-found.tsx</code> (about a day). Delete dead versions. Re-enable type checking
2	API proxy	Does not pass the caller's address to the backend, so every user looks like one client . 93% of access records are meaningless	Forward <code>X-Forwarded-For</code> ; run the backend with <code>--proxy-headers</code> . Must precede any audit log
3	Backend	Unsupervised orphan — dies permanently, does not return after reboot. Logs to <code>/tmp</code> , erased on reboot. <code>/docs</code> open on the LAN. Directly exposed	systemd unit (<code>Restart=always</code> , pinned log path). <code>ENVIRONMENT=production</code> . Move to loopback behind nginx
4	PostgreSQL	No backup — a disk failure loses everything. Only ever seen 874 rows	Nightly encrypted dump, off-host, with a rehearsed restore . Load test at realistic volume
5	Redis	No authentication (low risk while loopback-only). Rate limiter connected but unused	Set a password. Actually apply rate limits to login and upload routes
6	NLP gateway	Unsupervised orphan (14 days). Exposed on the LAN	systemd unit. Move to loopback
7	NLP classifier container	Adequate — auto-restarts, loopback, stable for two weeks	Keep as-is

#	Component	What is missing today	What makes it production grade
8	AI watcher	Unsupervised orphan (13 days). If it stops, uploads silently go unprocessed with no alert	systemd unit. Alert on processing backlog
9	External services	No defined behaviour shown to users when Azure or REDCap is down	Define user-facing failure messaging and a retry policy
10	De-identification app	The deployed build is from 24 July, so recent fixes are not on clinical machines. Re-identification key line 2 is corrupt (SID_3 unreadable)	Rebuild and redeploy to clinical machines. Repair the key file by hand

What the diagram makes visible

- **Only the two containers (1, 7) restart themselves.** All three native processes (3, 6, 8) are PPID-1 orphans. This is why axis A comes first.
- **Identity is destroyed along the data path.** Browser → the proxy (2) strips the address → the backend (3) sees a single container. However well an audit log is built, it is worthless until component 2 is fixed.
- **Three components are LAN-exposed** (1, 3, 6). Only the webapp closes automatically after 8/15; the backend and the NLP gateway stay open.
- **PostgreSQL (4) is where everything terminates** and it has no backup. It is both the point with the most inbound arrows and the one whose loss cannot be undone.

A. Release and deployment — Blocker

Why this matters

The system is running because a person started it by hand in a terminal. If a program stops for any reason, **it stays stopped until somebody notices and starts it again**. After a reboot the website loads but no data appears — a state that is genuinely hard to diagnose.

What is needed is a **registration that says "if this stops, start it again; if the machine reboots, bring it back."** Linux already has this (systemd); it takes two configuration files. It is **the highest value-per-hour item in this document**.

The companion need is **version labelling**. Today nobody can answer "which version is on the server?" — which means "let's roll back to the version that worked" is not an available move.

No process supervision

Both long-running processes are **orphans reparented to PID 1**:

```
PID 2117291 PPID 1 up 12d15h uvicorn main:app --host 0.0.0.0 --port 18001 --workers 3
PID 1438747 PPID 1 up 12d23h main_complete_pipeline_db_api.py --config config_remote.yaml
```

They were started from a terminal that has since exited. There is **no systemd unit, no supervisor, no pm2** — verified absent at both user and system scope. The consequences:

- If either process dies — OOM, unhandled exception, a stray `kill` — **it stays dead**. Nothing notices, and per axis C nothing alerts.
- **On reboot, neither comes back**. The webapp container returns (`restart: unless-stopped`), so the site loads and then fails every API call.

The pipeline watcher's 12-day uptime is luck, not design.

No version identity

`git tag` returns **0 tags**; there are no GitHub releases. Nobody can answer "what version is in production?" and there is no known-good artifact to roll back to. The running image is identified only by a build timestamp.

No staging environment

Changes go from a developer machine to the production box. The `staging/caire` branch is a branch, not an environment.

Recommended

1. systemd user units for the backend and the watcher, with `Restart=always` and `WantedBy=default.target`, plus `loginctl enable-linger` so they survive logout and reboot. This is the single highest-value change in this document.
 2. Annotated git tags on every deploy; record the deployed tag somewhere the running system reports (e.g. `/health`).
 3. A documented rollback: previous tag, rebuild, restart — rehearsed at least once.
-

B. CI and quality gates — Blocker

Why this matters

CI is the machinery that checks, every time the code changes, that nothing else broke. This project **has that machinery**. The problem is that the nightly check has been **red for two months**.

When an alarm sounds every day, people learn to silence it. That is exactly the present state — 20 of the 21 checks pass every night, but one keeps failing, so the whole run shows red and **nobody looks at the result**. A genuine failure would look identical. **This is worse than having no alarm**, because it manufactures false confidence.

The fix is one line in one test. It is not hard, and once done, "red means a real problem" becomes a trustworthy signal again.

Separately: the build currently **passes even when the code contains outright errors**, because the checks were switched off in configuration. It is the equivalent of writing a paper with the spellchecker disabled.

The nightly E2E suite has never passed

Metric	Value
Runs examined	60
Successes	0
Failures	60
Oldest run	2026-06-16
Most recent	2026-08-13

Roughly two months of nightly runs, none green. The cause is a single test:

```
tests/e2e/test_full_flow.py:337 test_redcap_import_sample
    assert resp.status_code == 200
E assert 400 == 200
1 failed, 20 passed, 3 skipped, 512 deselected
```

The test asserts `200` unconditionally, but REDCap is not configured in CI, so the route correctly returns `400`. **This is a test defect, not a production bug** — neighbouring REDCap tests guard themselves with a skip and this one does not.

The defect is small. The situation is not: a red safety net that everyone has learned to ignore is worse than no safety net, because it produces false confidence. 20 other E2E tests pass every night and nobody can see that, because the run is red regardless.

Type and lint errors cannot fail a build

app/Webapp/next.config.js :

```
typescript: { ignoreBuildErrors: true },
eslint:      { ignoreDuringBuilds: true },
```

`npm run build` therefore does **not** function as a type check, contrary to reasonable assumption. There is also **no ESLint configuration file** in the repository at all — `next lint` drops into an interactive setup prompt rather than linting.

Other gaps

- No coverage threshold (`fail_under` unset in `pyproject.toml`).
- No dependency vulnerability scanning — no Dependabot, `pip-audit`, or `npm audit` in CI.

Adequate today

`backend-ci.yml` and `webapp-ci.yml` exist, are path-filtered, run on every branch, and pass on `staging/caire`. The `pre-push` hook (ruff + pytest collection) works and has caught real problems.

Recommended

1. Fix `test_redcap_import_sample` to skip when REDCap is unconfigured, then **keep the nightly run green** and treat red as actionable.
2. Remove both `ignore*` flags and fix whatever surfaces. Add an ESLint config.
3. Add `pip-audit` and `npm audit` to CI; enable Dependabot.
4. Set a coverage floor at the current measured level so it cannot regress.

C. Observability — Blocker

Why this matters

Observability is, in plain terms, **the dashboard**: can you tell from outside whether the system is healthy, and where it hurts?

There is no dashboard. Errors are not recorded anywhere, slowdowns are invisible, and **above all, nothing contacts a human when something breaks.**

The proof that this is not hypothetical is axis B: the nightly check **failed for two months and nobody knew**. With alerting it would have been known on day one. By the same token, if the backend stopped right now, the first signal would be a patient reporting that the page will not open.

Logs are a related problem. Nothing has ever cleaned them, so files from November 2025 are still there. This server is shared between research groups, so a full disk stops other people's projects too.

Nothing in this axis exists.

Capability	State
Error tracking (Sentry or equivalent)	Absent
Metrics / APM (Prometheus, OpenTelemetry)	Absent
Alerting	Absent
Request correlation IDs	Absent
Log rotation	Absent

The two-month CI failure is the proof. A system with alerting would have surfaced it on day one.

Logging does not match its documentation

`core/logging.py` uses plain `logging.basicConfig`. It does **not** use structlog — while `app/Backend/CLAUDE.md` states "structlog is configured in `core/logging.py`". A new contributor reading that will look for structured output that is not there.

Without correlation IDs, a request cannot be followed from the webapp proxy through to the backend handler, which makes incident diagnosis guesswork.

Logs grow without bound

`app/Backend/Logs/` holds **37 MB**, with files dating back to `compose-20251120-143726.log` — November 2025, never pruned. Nothing rotates or expires. On a shared host this eventually becomes someone else's problem too.

Worse, that directory is **not where the running backend writes**. `uvicorn_native.log` there stopped being written on 14 July; the live process logs into a temporary scratch directory under `/tmp` that a reboot will erase. See axis J.

Recommended

1. Alerting first, and route it somewhere a human reads. Start with the nightly CI result and a liveness check on the two supervised units.
 2. `logrotate` for the log directories with a defined retention that matches the PHI retention policy (axis E).
 3. Correlation ID middleware, propagated from the Next.js proxy.
 4. Either adopt structlog or correct `CLAUDE.md`. Documentation that lies is a defect.
 5. Error tracking (self-hosted, given PHI and BAA constraints — a hosted SaaS would need its own agreement).
-

D. Reliability — Gap

Why this matters

This axis is **in better shape than expected**. The system checks its own health properly (`/health`), and outbound calls carry time limits so nothing waits forever. That is better than most projects at this stage.

What remains is that **there is exactly one of everything** — one server, one pipeline, one classifier. At this research scale that is a reasonable choice, and multiplying them is not necessary.

The catch is that without the automatic restart from axis A, **one component stopping means a permanent outage**. Put the other way round: fixing axis A largely fixes this axis for free, because the health checks needed to drive a supervisor and an alert already exist.

Adequate today

- `/health` genuinely probes the database and Redis and degrades to `unhealthy` rather than 500-ing; `/ready` is a separate, cheaper check. This is better than most projects at this stage.
- Outbound calls carry timeouts: REDCap 60 s, NLP 30 s, patient lookup 10 s.
- The webapp container has a working healthcheck and `restart: unless-stopped`.

Gaps

Every component is a single instance: one watcher, one NLP container, one backend host, one database. That is a reasonable choice at this scale — but combined with the absence of supervision (axis A) it means any single failure is **silent and permanent** until a human notices.

Nothing to fix here beyond axis A; the health checks are already good enough to drive a supervisor and an alert.

E. Data and backup — Blocker

Why this matters

There is no backup at all. A failed disk, or one mistyped command, and every pipeline result, survey response, and behavioural record is **gone permanently**. This is research data, so it cannot be re-collected — you cannot ask patients to sit through the same consultation again.

One common trap deserves naming: **a backup you have never restored is not a backup**. Backup files appearing nightly while the restore silently fails is a genuinely frequent outcome. That is why "one rehearsed restore" is written into the plan.

The other gap is **scale experience**. The largest table today holds 874 rows; a running study will produce tens or hundreds of thousands. Nobody has checked whether pages stay fast at that size. Discovering it after patient numbers grow makes it far harder to fix.

No backups

`crontab -l` contains **no backup job**. There is no dump, no snapshot, no off-host copy. A disk failure or an accidental `DROP` loses every pipeline result, survey response, and behavioural log permanently.

Never exercised at production volume

The largest table holds **874 rows**:

Table	Rows
<code>nlp_all_predictions</code>	874
<code>patient_followup_survey_page_behavior</code>	269
<code>doctor_behavior</code>	214
<code>patient_survey_submission_log</code>	214
<code>session_recording</code>	147

Load testing amounts to one file, `load_tests/test_100_doctors.py`. Query plans, pool sizing, and page render times are all unvalidated at realistic scale.

The re-identification key is corrupt and unrepaired

`AI_physician_patient_communication/data/deid_mapping.csv` line 2 holds 21 fields because a write lost its line terminator, gluing one record's `doctor_link` to the next record's `real_sid`. `SID_3` is consequently unparseable — and that row is the only record of a real visit date. A guard was added upstream so a migration refuses rather than silently dropping it, **but the file itself is still broken**.

Session replays grow without a retention rule

`session_recording` holds 148 rows / 2,930 kB of gzipped screen replay covering 13 days, with no expiry rule and no classification as PHI even though it reproduces the patient's screen. Detail in axis J.

Adequate today

Alembic is at head (`034_widen_speaker_columns`) with 34 revisions applied cleanly, and all 17 tables carry indexes.

Recommended

1. Nightly `pg_dump`, encrypted, stored off-host. **It is not a backup until a restore has been rehearsed.**
 2. Repair `deid_mapping.csv` line 2 by hand, from whatever source can reconstruct `SID_3`.
 3. Define data retention and deletion — required by policy, and it also bounds the log problem in axis C.
 4. Seed a copy of the database at realistic volume and re-run the load test before go-live.
-

F. Frontend production readiness — Gap

Why this matters

This is about **what the patient sees when something goes wrong on screen.**

Today, an error in a report page shows the developer crash screen. A patient who followed a link from their clinician to read their own consultation results is met with an English error message and fragments of code. A single **"Something went wrong — please contact your care team"** page fixes it, and it is roughly a day of work.

The second issue is **maintainability**. The interface is 99 files and 151,000 lines, with 62 older versions of the same screens sitting alongside the live ones. Working out which file is actually in use means tracing by hand, which **makes even small changes risky**. It causes no outage today; it makes every future change slower and more dangerous.

No error or loading boundaries

The App Router contains **zero** `error.tsx`, `global-error.tsx`, `not-found.tsx`, or `loading.tsx` files. Any render exception in a patient-facing report therefore shows the **default Next.js error screen** — a stack trace in development, an unstyled generic failure in production. For a patient opening a link from their clinician, this is the worst possible failure mode, and it is roughly a day's work to fix.

The component layer has outgrown its conventions

Metric	Value
<code>.tsx</code> components	99
Total lines	151,333
Files over the repo's own 150-line limit	89
Largest file	<code>ReportDownload.tsx</code> — 8,178 lines

`CLAUDE.md` specifies "component files < 150 lines". Ninety percent of components violate it, several by two orders of magnitude.

Version proliferation obscures what is live

Prefix	Files
PhysicianReportsModifiedV*	23
PatientReportModifiedV*	17
ConsultationScoringV*	8
PatientInitialVisitReportV*	7
PatientFollowUpReportV*	7

Sixty-two versioned variants of five components. Determining which one a given route actually renders requires tracing imports by hand — which makes every change riskier than it should be, and makes dead code indistinguishable from live code.

A convention violation worth naming

`PhysicianReportsModifiedV41Timothy.tsx` embeds a person's given name in a filename. `CLAUDE.md` absolute rule 4 forbids real collaborator names in source. It is referenced from two places, so removing it is a rename plus two import updates.

The proxy discards the client's identity

`src/app/api/backend/[...path]/route.ts:18` constructs a fresh header object and forwards only the API key, authorization, and content type. The client's `X-Forwarded-For` never reaches the backend, which is why 93% of access-log entries name the container rather than a user. Detail in axis J.

Not assessed

No accessibility audit has ever been run against a patient-facing medical interface. This is likely an institutional requirement as well as a good idea.

Recommended

1. Add `error.tsx`, `global-error.tsx`, and `not-found.tsx` — highest value per hour in this document after supervision.
2. Identify which component versions are reachable, delete the rest, and stop the versioned-filename pattern in favour of git history.
3. Rename the file carrying a personal name.
4. Run an a11y audit (axe or Lighthouse) on the three patient-facing routes.

G. Testing — Gap

Why this matters

The fundamentals here are **good**. There are 604 backend tests, and they were migrated to run against the same kind of database production uses, which makes them trustworthy. Testing against a simplified stand-in database — where tests pass but reality fails — is a common trap this project has already avoided.

What is missing is that **the whole-journey check (E2E) is dead** (axis B). Individual parts are tested well, but the scenario "a patient opens their link and completes the survey end to end" has been ignored for two months. That check exists precisely to catch the case where every part works and the assembly does not.

Suite	Count	State
Backend (<code>pytest</code>)	604 (581 non-E2E)	Runs against real PostgreSQL
Webapp (<code>jest</code>)	27	Passing
E2E (Playwright)	6 specs	Exists
E2E (backend, nightly)	21	Red for 60 consecutive runs
Load	1 file	Never run at scale

The backend suite is genuinely good — it was migrated to real PostgreSQL, so foreign keys, column limits, and transaction semantics are actually enforced rather than papered over by SQLite. The weakness is not unit coverage; it is that the integration safety net is broken (axis B) and the frontend has no coverage floor.

H. Documentation and operational procedure — Gap

Why this matters

The installation documentation is good. What is absent is documentation for **what to do when something goes wrong**.

That knowledge currently lives only in the head of the person who built the system. If they are on leave or move on, whoever remains does not even know where to look when the backend dies overnight. **A single page of response steps** changes that situation completely.

The one that matters most is a **procedure for a reported patient-data exposure**. HIPAA requires it, and it cannot be written after the incident has already happened.

Adequate today

`docs/setup/` carries six deployment documents including `NETWORK_EXPOSURE.md`, and `docs/INDEX.md` catalogues them.

Missing

Searching `docs/` for runbook, incident, rollback, or monitoring material returns **zero** documents. There is no written answer to:

- The backend is down at 2 a.m. — what does the on-call person do?
- A deploy went wrong — how is it reversed?
- A patient reports seeing another patient's data — what is the response procedure? (HIPAA requires one.)

`CLAUDE.md`'s structlog claim is factually wrong (axis C), which is a documentation defect in its own right.

I. Security — Blocker

Why this matters

Real patient information means HIPAA applies. Three items matter most in the current state:

First, traffic is not encrypted (no HTTPS). Patient reports and survey answers cross the network **in the clear**. Anyone on the same network who chooses to look can read them. The padlock icon in a browser's address bar is precisely this protection.

Second, the API documentation page is open. One configuration line has left a developer screen switched on, so anyone on the network can browse the complete list of what this system can do. Changing that one value closes it — no code change needed.

Third, there is no record of who opened which patient file. HIPAA requires it explicitly. Read this together with axis J: not only is the record absent, **there is no way to reconstruct it afterwards**.

The admin password (`admin1234567`) also needs replacing. It is stored using a method that is weak by current standards, so a database leak would reveal the original almost immediately.

Full detail was assessed separately; the findings are summarised here and the remediation is folded into the phased plan below.

Finding	Detail
No TLS	Webapp <code>3001</code> and backend <code>18001</code> both plain HTTP
<code>/docs</code> exposed	<code>.env</code> says <code>ENVIRONMENT=development</code> , so <code>/docs</code> and <code>/openapi.json</code> return 200 from the LAN
No PHI audit log	Nothing records who read which patient record — HIPAA § 164.312(b)
Shared API key = superuser	One key; any caller presenting it gets <code>is_superuser=True</code> (<code>auth/backends/api_key.py:33-38</code>)
Weak admin credentials	<code>admin1234567</code> , hashed with single-round salted SHA-256 (<code>auth/admin_routes.py:48-64</code>)
Backend on <code>0.0.0.0</code>	<code>scripts/run-backend.sh:60</code> — the webapp can be bypassed
No login protection	No rate limit, lockout, or failure logging on <code>/api/admin-auth/login</code>
No security headers	No HSTS, CSP, X-Frame-Options, or nosniff
Rate limiter unused	Initialised in <code>app_lifespan.py</code> , applied to no route

Finding	Detail
Re-identification key in plaintext	<code>deid_mapping.csv</code> mode <code>664</code> , on the same host as the data it re-identifies — see <code>PHI_COMPLIANCE.md</code> § 3

Adequate: PostgreSQL and Redis are loopback-only, uploads are capped at 25 MB, login messages do not enable account enumeration, and `NEXT_PUBLIC_API_KEY` has been removed.

The structural constraint

This host is a **shared multi-user machine with more than ten accounts**. Root and every sudo user can reach the PHI, and no application change prevents that. Confirming the IRB scope and the BAA with institutional IT is a prerequisite, not a follow-up. **This plan can satisfy the technical safeguards; it cannot make "HIPAA compliant" a claim this server asserts alone.**

J. Access logging and retention — Blocker

Why this matters

A hospital system must retain a record of **who opened which patient file, and when**. It is a legal requirement, and it is the only way to establish the scope of an incident after one occurs.

That record is not being kept. But there is a subtler problem underneath. Access records are being written — and **every one of them shows the same address**. Because all users arrive through an intermediate program, the system sees 12,000 accesses as a single visitor. **The log exists; the person does not.**

The file holding those records also sits in a **temporary folder** (`/tmp`). One reboot erases every access record collected so far. Meanwhile the folder developers would naturally check has been empty since 14 July, so anyone who looks concludes there is no access log at all.

Order matters here. Building the audit feature first would produce ten thousand rows all naming the same address. Fixing the intermediate program so it passes the real visitor's address must come **first**.

The question this axis answers is: **if someone accessed patient data yesterday, can you find out who?** Today the answer is no — and not merely because nothing archives the logs. The data needed to answer it is never captured.

The access log records the wrong address

The backend does write an access log. Of 12,071 logged requests:

Source address	Requests	What it actually is
172.31.0.2	11,248 (93%)	The Docker webapp container
127.0.0.1	821	Loopback
10.226.8.205	2	A real LAN client

All browser traffic reaches the backend through the Next.js proxy, so the backend sees one container address for every user. The cause is in the proxy, which builds a fresh header object rather than passing the request's own headers through:

```
// app/Webapp/src/app/api/backend/[...path]/route.ts:18
const headers: HeadersInit = {};          // starts empty
headers["X-API-Key"] = API_KEY;           // only these are set
headers["Authorization"] = ...;
headers["Content-Type"] = ...;
```

Neither `X-Forwarded-For` nor `X-Real-IP` is forwarded. **The log exists; the identity does not.**

The log lives in `/tmp`

```
/tmp/claude-1007/-home-choih2-test-prostate-cancer/
<session-id>/scratchpad/backend_18001.log
2.7 MB · 30,844 lines · 12,071 access entries
```

`/proc/2117291/fd/1` resolves to that path: the backend was started on 31 July from a tooling session whose stdout was redirected into a temporary scratch directory. Consequences:

- **A reboot erases it** — it is under `/tmp`.
- Nobody would look there. The in-repo `app/Backend/logs/uvicorn_native.log` stopped being written on 14 July, so anyone checking the obvious location concludes there is no access log at all.

This is a direct consequence of the missing process supervision in axis A: a systemd unit would have pinned the log destination.

Nothing rotates, compresses, or archives

Check	Result
<code>logrotate</code> config for this project	None
<code>*.log.gz</code> or <code>*.log.N</code> files anywhere	0
Log archival script	None
<code>app/Backend/logs/</code>	37 MB, files back to November 2025

Behaviour tracking knows the session, not the person

The three tracking tables carry:


```
session_id, file, speaker, event_type, event_metadata,
device_type, client_timestamp, created_at
```

No IP address, no user agent, no user identifier. They record what happened within a session, never who did it or from where. They are analytics, not an audit trail, and cannot be retrofitted into one without a source address to attach.

The one thing that is compressed and stored

`session_recording` holds rrweb session replays gzipped into a `LargeBinary` column:

```
148 rows · 2,930 kB compressed · 2026-07-31 to 2026-08-13 (13 days)
```

This is the closest existing asset to "who did what". Three problems:

- It exists for **UX replay**, not access auditing — there is no identity or address field.
- **No retention policy.** Thirteen days have accumulated with no expiry rule, so it grows without bound.
- It replays the patient's screen pixel for pixel, which makes it **PHI in practice** — but it is not treated as such under `PHI_COMPLIANCE.md` § 3, and it is stored unencrypted in a column like any other.

No PHI access audit log

HIPAA § 164.312(b) requires a record of who accessed which patient record. None exists (axis I). The findings above make that worse than a simple omission: **the gap cannot be closed retroactively from logs**, because the logs never held the answer.

Recommended

1. Pin the backend's log destination to the repository `logs/` directory — free if done alongside the systemd unit in axis A (`StandardOutput=append:`).
2. Forward `X-Forwarded-For` from the proxy and run uvicorn with `--proxy-headers --forwarded-allow-ips=127.0.0.1` . **Everything else in this axis depends on this**; without it an audit log records the container address 12,000 times.
3. `logrotate` with compression and a retention period matching the PHI policy.
4. Define retention and expiry for `session_recording` , and classify it as PHI.

Phased remediation

Ordered by what fails first, not by axis. Security is spread across phases rather than front-loaded, because a system that stops on reboot cannot be secured into working.

Phase 1 — Stop silent failure (days)

1. systemd user units with `Restart=always` for backend and watcher; `enable-linger`. Pin `StandardOutput=append:` to the repository `logs/` directory in the same unit, which also moves the access log out of `/tmp` at no extra cost (A, J)
2. Alerting on the two units plus the nightly CI result (C)
3. Fix `test_redcap_import_sample`; get the nightly run green (B)
4. Nightly encrypted `pg_dump` off-host, with one rehearsed restore (E)
5. `ENVIRONMENT=production` and `openapi_url=None` — closes `/docs` with no code risk (I)
6. Rotate the admin password; `chmod 600` the re-identification key and the archives (I)

Nothing here requires a certificate, a reboot, or institutional sign-off. Together they remove every failure mode that is currently both silent and permanent.

Phase 2 — Make failure visible and reversible (1-2 weeks)

1. `logrotate` with compression and a defined retention (C, E, J) 7a. **Forward X-Forwarded-For from the proxy** and run unicorn with `--proxy-headers --forwarded-allow-ips=127.0.0.1`. Until this lands, every access record names the container instead of a user — which makes item 16 below worthless (J)
2. Correlation ID middleware; reconcile structlog with `CLAUDE.md` (C, H)
3. Git tags on deploy, plus a written rollback procedure (A, H)
4. `error.tsx` / `global-error.tsx` / `not-found.tsx` (F)
5. Remove `ignoreBuildErrors` / `ignoreDuringBuilds`; add an ESLint config (B)
6. Application hardening: config fail-closed, bcrypt password migration, login logout, apply the rate limiter, security headers, narrow CORS (I)

Phase 3 — TLS and the audit trail (2-4 weeks, partly external)

1. Obtain a certificate from institutional IT (the enquiry should be sent in Phase 1 — the wait is the long pole) (I)
2. nginx TLS termination; move webapp, backend, and the NLP gateway to loopback (I)

3. Per-user API keys (`AUTH_MODE=multi_key` , already implemented) — a prerequisite for a meaningful audit log (I)
4. `phi_access_log` table and a hook in `auth/access_control.check_patient_access` , the single gateway for patient data (I). **Depends on item 7a** — without forwarded client addresses this table records the proxy container for every row (J)

The nginx cutover is the most fragile step in this document. Dropping the backend to loopback makes it unreachable from the webapp container at `host.docker.internal:18001` . Verify on staging before applying.

Phase 4 — Sustainable engineering (1-2 months)

1. Repair `deid_mapping.csv` ; define data retention, including an expiry rule for `session_recording` and its classification as PHI (E, J)
2. Load-test at realistic volume (E)
3. Prune dead component versions; rename the file carrying a personal name; begin decomposing the largest components (F)
4. Dependency scanning; coverage floor (B)
5. Runbook, incident response, rollback docs (H)
6. Accessibility audit (F)

Phase 5 — Compliance, running throughout

1. Confirm the IRB-approved scope covers PHI on this shared host
 2. Confirm the BAA with institutional IT
 3. Documented risk analysis (§ 164.308(a)(1))
 4. Access provisioning and revocation procedure
 5. Retire or rewrite `SECURITY_AUDIT.md`
-

Verification

Each item is confirmed by observation, not by reading code.

Phase 1

```
systemctl --user is-enabled compass-backend compass-watcher # expect: enabled
systemctl --user restart compass-backend && sleep 5 && curl -s localhost:18001/health
sudo reboot # then confirm both units and the webapp return unattended
gh run list --workflow="Nightly E2E" --limit 3 # expect: success
curl -s -o /dev/null -w "%{http_code}\n" http://10.226.8.205:18001/docs # expect: 404
pg_dump ... && restore into a scratch database && compare row counts
```

Phase 1, log destination — `ls -l /proc/$(pgrep -f "uvicorn main:app")/fd/1` must resolve into the repository `logs/` directory, not `/tmp`.

Phase 2 — kill the backend by hand and confirm an alert arrives; force a component to throw and confirm a styled error page, not the Next.js default; run `npm run build` against a deliberate type error and confirm it now fails.

Phase 2, client address — load a patient page from a second machine, then grep the access log for that machine's LAN address. Finding it proves the forwarding works; seeing only `172.31.0.2` means it does not.

Phase 3

```
curl -sI https://<host>/ | grep -i strict-transport-security
ss -tlnp | grep -E ':(3001|18001|18080)' # expect: all 127.0.0.1
nmap --script ssl-enum-ciphers -p 443 <host> # expect: no TLS 1.0/1.1, no weak ciphers
```

Then a full regression pass: doctor dashboard, patient report, survey submission, admin upload — all over HTTPS.

Phase 3, audit log — open one patient report and confirm by SQL that `phi_access_log` gained exactly one row with actor, IP, and patient reference populated.

Throughout — `cd app/Backend && pytest -m "not e2e"` (581 of 604 collected; 23 are E2E and deselected). When a change breaks a test, decide whether it is a production bug **before** touching the test.

What this assessment does not solve

- **Protection from root and sudo users** on a shared host. Not achievable at the application level.
- **Full-disk encryption at rest.** Needs root and a reboot; blocked on institutional IT.
- **Declaring HIPAA compliance.** The technical safeguards are achievable here; administrative and physical safeguards and the BAA live outside this repository.
- **Obtaining a TLS certificate.** Depends on an institutional IT response, which is why the enquiry belongs in Phase 1 even though the work lands in Phase 3.
- **The single-host architecture.** Every component remains a single point of failure. Supervision makes failures short instead of permanent; it does not make them impossible.